

Fundación Fulgor

PROCOM

Informe

Trabajo Práctico N°1: Python

Autor: Alfici, Facundo Ezequiel

Documento: 44012327

Docentes: Ariel Pola, Santiago Garaventta Pascual y Agustín Pesce

21 de mayo de 2026

Índice

1. Introducción	3
2. Desarrollo	3
2.1. Marco Teórico	3
2.1.1. Filtros FIR	3
2.1.2. Filtros RC y RRC	4
2.2. Comandos	4
2.3. Flujo de programa	6
2.4. Ejemplos de funcionamiento	7
2.4.1. Generación de filtros	7
2.4.2. Ploteo de Filtros en frecuencia	7
2.4.3. Ploteo de Filtros en tiempo	10
2.4.4. Ploteo Total	13
2.4.5. RC vs. RRC	14
3. Conclusión	16

1. Introducción

En este laboratorio se buscará realizar un programa en Python que permita utilizar la comunicación serie para enviar y recibir datos desde el puerto virtual de Windows, con los cuales se generarán y graficarán filtros.

Para ello se utilizará el script `com_serie.py`, que se encargará de enviar y recibir los datos desde el puerto. Estos datos se enviarán hacia el archivo `main.py`, el cual tiene la tarea de procesar lo recibido y, según el comando, realizar las distintas acciones propuestas por el enunciado.

También se importó la clase `RaisedCosineFilter`, que es proveniente del archivo `filtros.py`. Esta clase se encargará de la generación de los filtros, siendo invocada desde el script `main.py` cada vez que se reciba el comando correspondiente.

2. Desarrollo

Para comenzar con el desarrollo, se realizará un marco teórico sobre los filtros que se desean generar.

2.1. Marco Teórico

Como parte principal del trabajo, se utilizaron filtros Raised Cosine (RC) y Root Raised Cosine (RRC), los cuales forman parte de los filtros FIR, debido a que necesitan una respuesta al impulso finita (de N muestras) y una fase lineal.

2.1.1. Filtros FIR

En lo que se refiere a los filtros FIR (Finite Impulse Response), podemos describir su funcionamiento utilizando la siguiente expresión, donde tomamos en cuenta coeficientes de realimentación nulos tal que $(a_k = 0 \forall k)$

$$y[n] = \sum_{k=0}^N h[k] x[n - k] \quad (1)$$

donde se tiene que $h[k]$ es la respuesta al impulso.

Las propiedades más importantes de este tipo de filtros son:

- *Estabilidad absoluta*
- *Fase lineal*
- *Independencia de muestras pasadas*

2.1.2. Filtros RC y RRC

El filtro ***Raised Cosine (RC)*** es la solución canónica al criterio de Nyquist, cuyo ancho de banda podemos manejar en base a un parámetro llamado roll-off(β), el cuál responde a la siguiente expresión.

$$W_{RC} = \frac{1 + \beta}{2T} \quad (2)$$

Una de las características más importantes de este filtro es la propiedad de **Interferencia Inter Símbolo (ISI)** nula. Donde, para cada $t = nT$, se tiene una ISI nula.

Respecto al filtro ***Root Raised Cosine (RRC)***, se utiliza para sistemas de comunicaciones reales, específicamente en sistemas donde se debe adaptar el transmisor Tx con el receptor Rx, de tal manera que se tiene que:

$$\boxed{H_{RRC}(f) * H_{RRC}(f) = H_{RC}(f)} \quad (3)$$

Pues una de las características más importantes de este filtro es que:

$$H_{RRC}(f) = \sqrt{H_{RC}(f)} \quad (4)$$

Donde se puede aprovechar las características mas beneficiosas del filtro ***RC*** y tener sistemas adaptados al mismo tiempo.

Algo importante a destacar antes de pasar a la siguiente sección, es definir los conceptos de los parámetros que se deberán cargar a los filtros, estos parámetros se describen a continuación.

- **alpha** — Representará el factor de **roll-off**. Es un float que debe tomar valores entre 0 y 1.
- **span** — Se refiere a la duración total del filtro expresado en cantidad de símbolos. Es decir, sabiendo que la respuesta al impulso se extenderá un tiempo determinado, este parámetro representa ese tiempo traducido a símbolos.
- **sps** — También llamado **Samples Per Symbol**, representa sobre las muestras por símbolo, donde cada símbolo representará a la señal como una forma de onda o como un punto (según si es TD o TC). Cuanto mayor sean las SPS, más precisa será la representación de la respuesta al impulso.

2.2. Comandos

A continuación, se explicarán los diferentes comandos que se implementaron en el script `main.py`.

- `generate_filter:(alpha, span, sps, rrc)` — Genera un filtro con los parámetros indicados.

`alpha` (*float*) — Factor de roll-off. Valor entre 0 y 1.

`span` (*int*) — Duración total del filtro en símbolos.

`sps` (*int*) — Muestras por símbolo.

`rrc` (*bool*) — Si es 't', se genera un filtro de raíz de coseno realzado. Si es 'f' o cualquier otro, se genera un filtro de coseno realzado.

- `plot_filter_freq:(filter_n, Continuous)` — Grafica un filtro específico en frecuencia.

`filter_n` (*int*) — Número de filtro a graficar. El número se asigna en orden de generación, comenzando por 0.

`Continuous` (*str*) — Si es 'c' o '' (vacío), se grafica en TC. Si es 'd' o cualquier otro, se grafica en TD.

- `plot_filter_time:(filter_n, Continuous)` — Grafica un filtro específico en tiempo.

`filter_n` (*int*) — Número de filtro a graficar.

`Continuous` (*str*) — Si es 'c' o '' (vacío), se grafica en TC. Si es 'd' o cualquier otro, se grafica en TD.

- `plot_filter:(filter_n, Continuous)` — Grafica un filtro específico en tiempo y frecuencia.

`filter_n` (*int*) — Número de filtro a graficar.

`Continuous` (*str*) — Si es 'c' o '' (vacío), se grafica en TC. Si es 'd' o cualquier otro, se grafica en TD.

- `plot_all_time:(Continuous)` — Grafica todos los filtros generados en el dominio del tiempo.

`Continuous` (*str*) — Si es 'c' o '' (vacío), se grafican en TC. Si es 'd' o cualquier otro, se grafican en TD.

- `plot_all_freq:(Continuous)` — Grafica todos los filtros generados en el dominio de la frecuencia.

`Continuous` (*str*) — Si es 'c' o '' (vacío), se grafican en TC. Si es 'd' o cualquier otro, se grafican en TD.

- `plot_simult_time:(Continuous)` — Grafica todos los filtros generados en el mismo gráfico en el dominio del tiempo.

`Continuous (str)` — Si es 'c' o '' (vacío), se grafican en TC. Si es 'd' o cualquier otro, se grafican en TD.

- `plot_simult_freq:(Continuous)` — Grafica todos los filtros generados en el mismo gráfico en el dominio de la frecuencia.

`Continuous (str)` — Si es 'c' o '' (vacío), se grafican en TC. Si es 'd' o cualquier otro, se grafican en TD.

- `plot_simult_all:(Continuous)` — Grafica todos los filtros generados en el mismo gráfico en el dominio del tiempo y la frecuencia.

`Continuous (str)` — Si es 'c' o '' (vacío), se grafican en TC. Si es 'd' o cualquier otro, se grafican en TD.

- `exit` — Termina la comunicación con el puerto serie.

2.3. Flujo de programa

Se realizó un diagrama de flujo que representa la forma en la que se moverá la información a la hora de ejecutar el script.

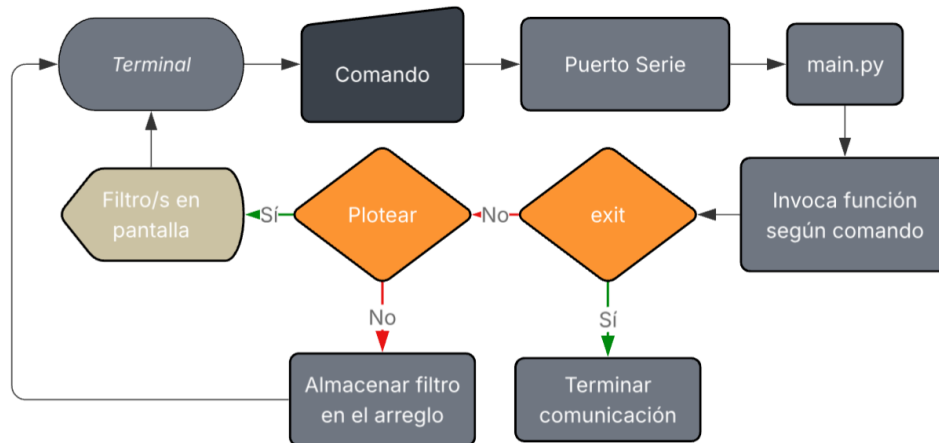


Figura 1: Diagrama de flujo

2.4. Ejemplos de funcionamiento

Lo que respecta a los ejemplos de funcionamiento, se utilizará un ejemplo por cada tipo de comando disponible para el usuario.

2.4.1. Generación de filtros

Para el manejo de la creación de filtros mediante el puerto serie, se enviará el siguiente comando por la terminal, hacia el puerto serie.

```
Terminal
1 generate_filter:0.5,10,10,t
```

Donde definimos un filtro con las siguientes características:

- $\alpha = 0.5$
- $\text{span} = 10$
- $\text{sps} = 10$
- $\text{rrc} = \text{'True'}$

Posteriormente a la primer generación, se generarán 2 filtros más para demostrar el funcionamiento de la herramienta de ploteo.

```
Terminal
1 generate_filter:0.9,30,100,t
2 generate_filter:0.1,10,5,t
```

2.4.2. Ploteo de Filtros en frecuencia

Como se explicó anteriormente en este informe, los comandos disponibles para graficar la respuesta al impulso en el dominio de la frecuencia permiten graficar un filtro a la vez o varios en simultáneo. Sabiendo esto, se colocó en la terminal:

```
Terminal
1 plot_filter_freq:0
```

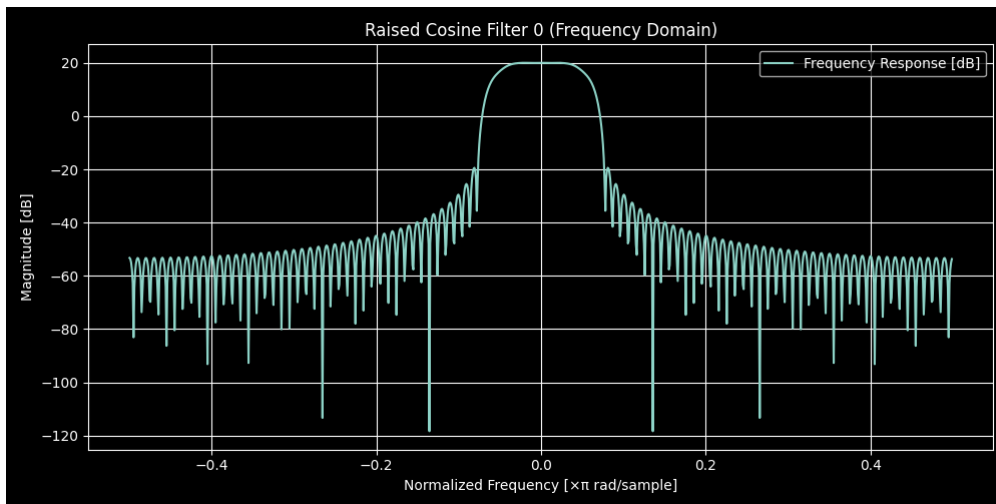


Figura 2: Filtro 0 en frecuencia y TC

Como el orden de generación de filtros define el número asociado al mismo, se graficará el filtro 0 en TC.

Dada la capacidad del comando, se generará el mismo gráfico pero en TD.

Terminal

```
1 plot_filter_freq:0,d
```

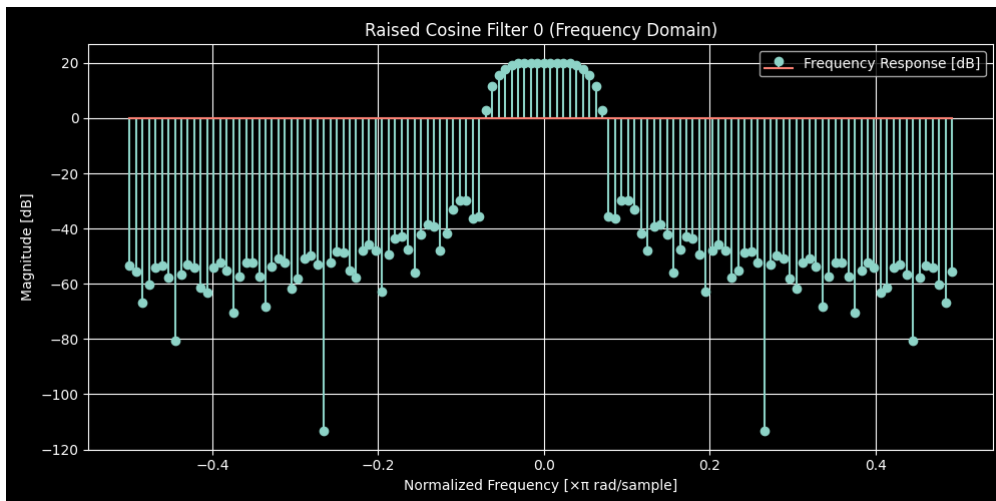


Figura 3: Filtro 0 en frecuencia y TD

Para finalizar con este inciso, se hará uso del comando que permite graficar varios filtros a la vez (Usando tiempo continuo en este caso).

```
Terminal
1 plot_all_freq:c
```

Donde se obtuvo la siguiente figura.

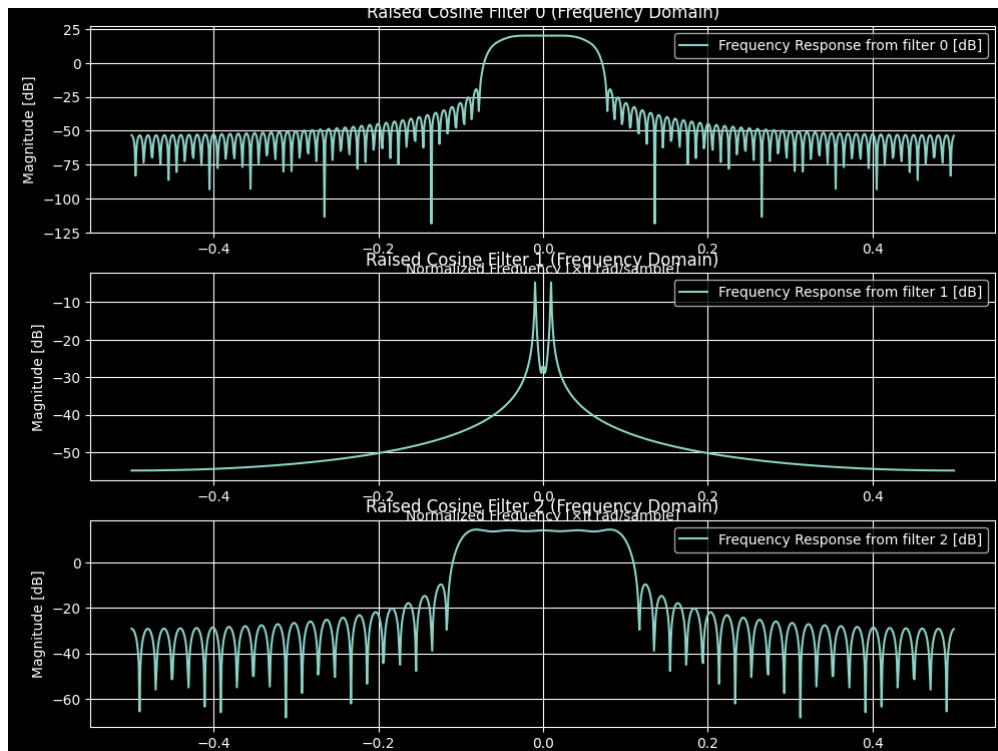


Figura 4: Filtros 0, 1 y 2 en frecuencia y TC.

Algo clave que se puede observar en la Figura 4 es que, para los filtros con un α más cercano a 0, la forma del espectro tiende a ser más rectangular, es decir, presenta un ancho de banda mayor. Por otra parte, en los filtros con un α más cercano a 1 la forma del espectro tiende a ser más suavizada, lo que limita el ancho de banda.

Este fenómeno puede verse con mayor claridad utilizando el siguiente comando.

Terminal

```
1 plot_simult_freq:c
```

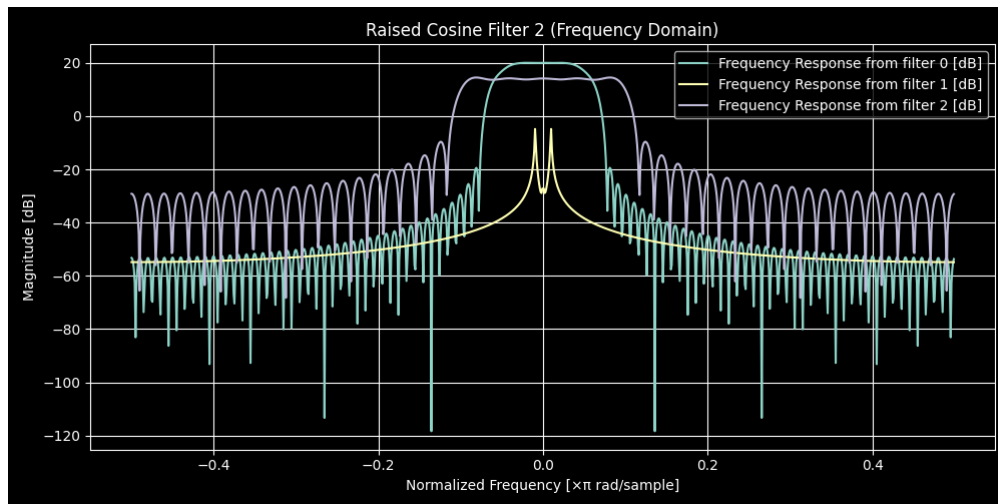


Figura 5: Filtros 0, 1 y 2 superpuestos en frecuencia.

2.4.3. Ploteo de Filtros en tiempo

Para el dominio temporal, se tienen las mismas funciones que se utilizaron para inciso anterior, con lo cual podemos graficar la respuesta al impulso del filtro 0.

Terminal

```
1 plot_filter_time:0,c
```

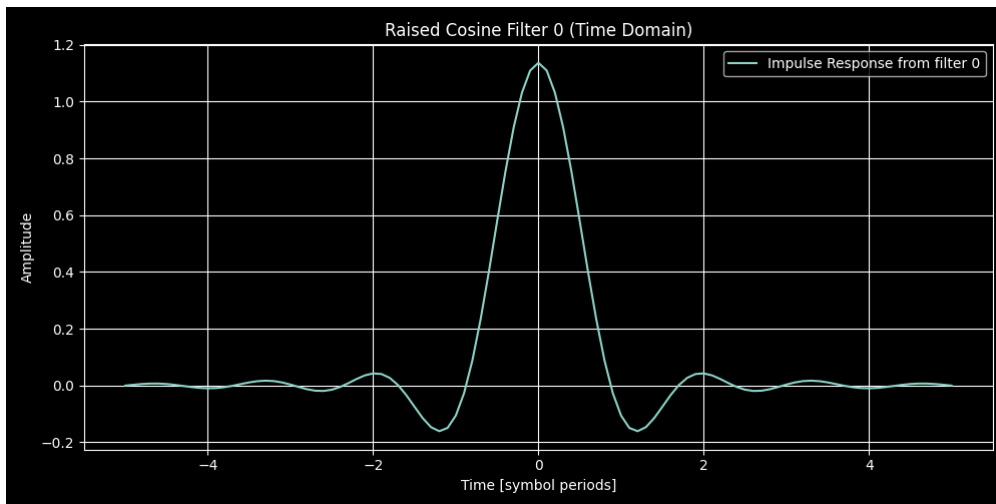


Figura 6: Filtro 0 en el dominio temporal y TC.

Para un tiempo continuo, se observa la Figura 6.
De igual manera, se puede realizar para tiempo discreto.

Terminal

```
1 plot_filter_time:0,d
```

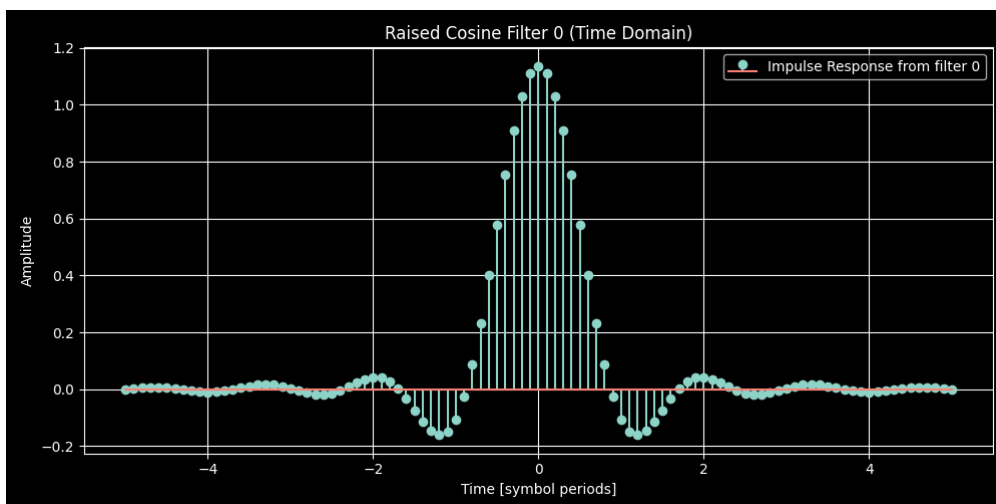


Figura 7: Filtro 0 en el dominio temporal y TD.

Posteriormente, se realizará el ploteo de las respuestas al impulso de todos los

filtros en diferentes subplots.

Terminal

```
1 plot_all_time:c
```

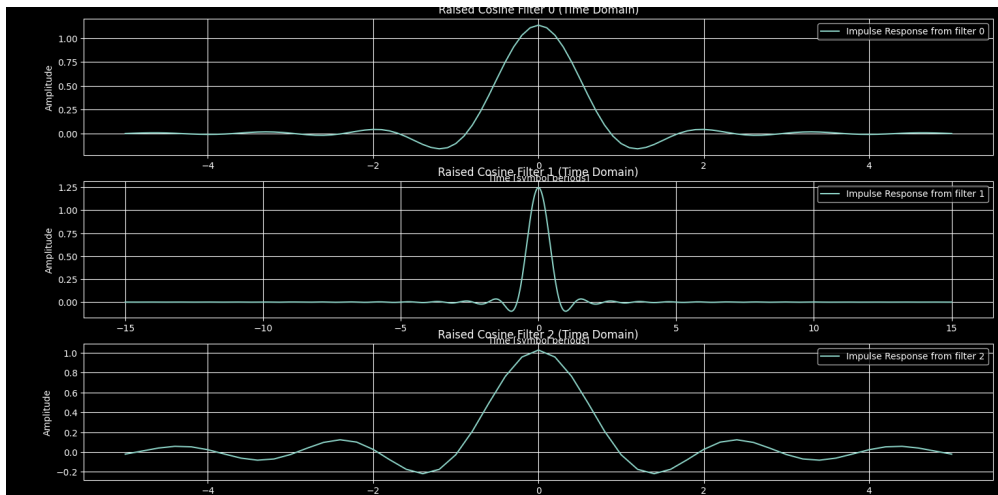


Figura 8: Filtros 0, 1 y 2 en el dominio temporal y TC.

Como ya indicaba la frecuencia, para un valor de **alpha** cercano a 0, se tiene una respuesta temporal más suavizada, mientras que para un valor de **alpha** cercano a 1, la respuesta temporal es mucho mas abrupta.

Por otro lado, en lo que refiere a las muestras por símbolo, la diferencia clara se puede visualizar utilizando el comando que nos permite superponer las respuestas.

Terminal

```
1 plot_simult_time:c
```

Para lo que se obtiene la siguiente figura.

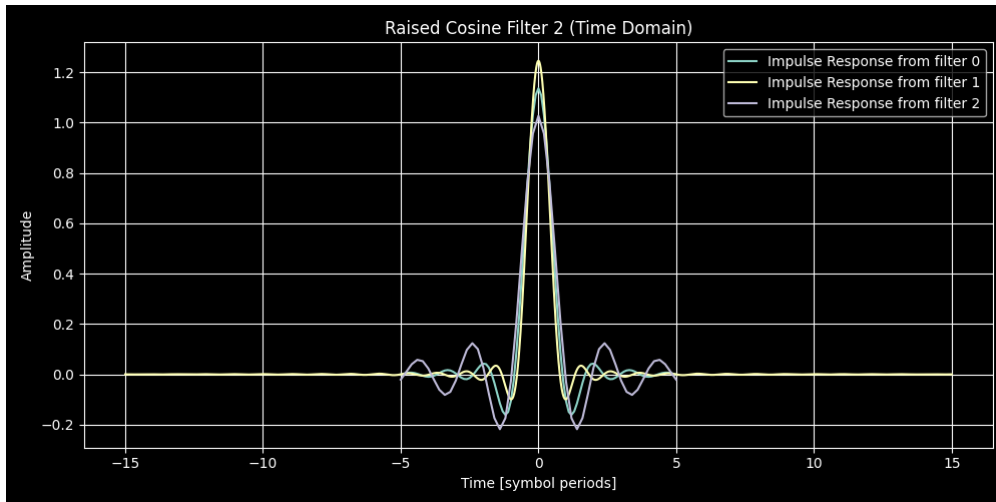


Figura 9: Filtros 0, 1 y 2 superpuestos en tiempo.

Donde se ve claramente que, para un valor de muestras por símbolo mayor, la curva se representa de una manera mucho más suavizada y representativa de lo que sería una respuesta de un sistema real.

2.4.4. Ploteo Total

En caso de querer visualizar las respuestas al impulso y la respuesta en frecuencia de todos los filtros utilizados como ejemplo, se puede utilizar el siguiente comando.

```
Terminal
1 plot_simult_all:c
```

Donde se generará la gráfica que muestre el apartado en frecuencia y en tiempo de todos los filtros superpuestos entre sí, como se ve a continuación.

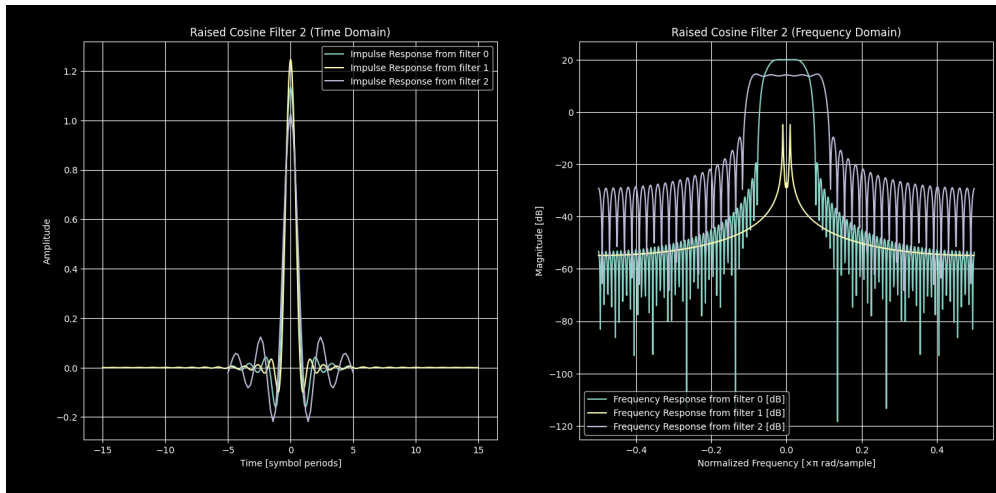


Figura 10: Filtros 0, 1 y 2 superpuestos en tiempo y frecuencia.

2.4.5. RC vs. RRC

Finalizando con la parte de análisis, se generarán dos filtros muy parecidos, pero con una diferencia; Uno será un filtro Raised Cosine y el otro será un filtro Root Raised Cosine.

El objetivo de este experimento es visualizar de manera clara las diferencias entre un tipo de filtro y otro para un caso en el que ambos presentan los mismos parámetros.

Se colocará por la terminal los siguientes comandos.

```
Terminal
1 generate_filter:0.5,10,10,t
2 generate_filter:0.5,10,10,f
3 plot_simult_all:c
```

Entonces, se generó:

- **Filtro 0:**(alpha=0.5,span=10,sps=10,RRC=True)
- **Filtro 1:**(alpha=0.5,span=10,sps=10,RRC=False)
- **Plot simultáneo de tiempo y frecuencia**

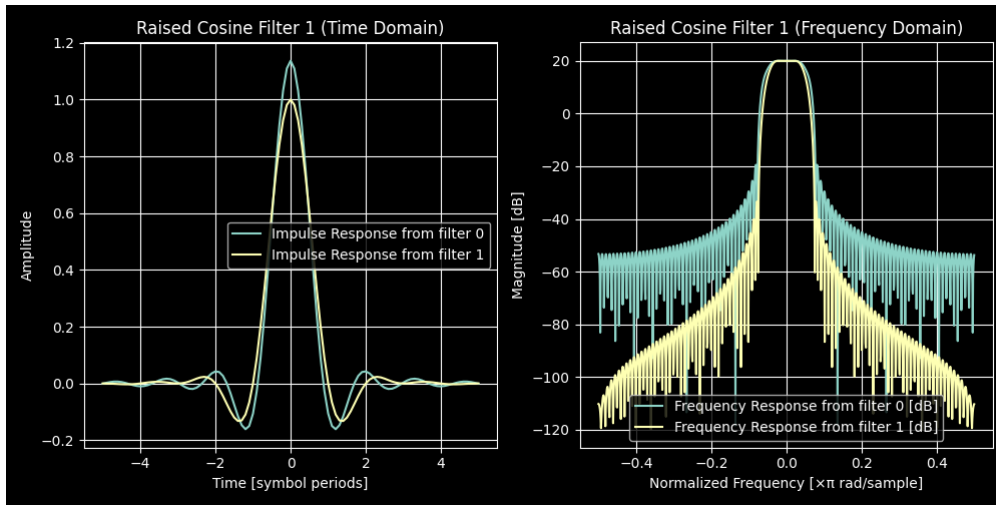


Figura 11: Filtros RC y RRC superpuestos en tiempo y frecuencia.

Se confeccionó una tabla comparativa para ver los resultados.

Dominio	Filtro RC	Filtro RRC
Tiempo	Ceros en $k * T$	No hay ceros en $k * T$
Frecuencia	Ajuste de cero ISI	Pérdidas en bandas no útiles

Cuadro 1: Comparación de tipos de filtro

El apartado más significativo fue el de la frecuencia, donde se puede ver perfectamente el efecto de la ISI nula. A medida que se aleja la frecuencia de la banda central, se atenúan las pérdidas en las bandas no deseadas.

3. Conclusión

Se cierra este informe concluyendo que, en lo que respecta a **Python**, la capacidad del lenguaje para el manejo de datos es extremadamente útil. Las posibilidades que nos permiten los scripts es ilimitada en el área del procesamiento de información, lo cuál será de gran ayuda en caso de utilizar la comunicación en serie para laboratorios posteriores.

La dificultad principal fue adaptar correctamente la comunicación serial con el script principal, debido a malinterpretaciones de la consigna, lo cual se solucionó modificando el script para que en vez de que se haga en bucle, se invoque al declararse como función.